

LTMS — API Design ตอนที่ 5

OpenAPI 3.1 Specification

เวอร์ชัน: 0.1 (ร่างแรก) วันที่: 2 สิงหาคม 2569 ต่อจาก: LTMS_API_Design_Part3.md + Part3-1.md (schema) และ Part4.md (error catalog) **ไฟล์ผลลัพธ์:**

- openapi/LTMS_OpenAPI_MVP.yaml — 93 endpoint (MVP/R01)
- openapi/LTMS_OpenAPI_Sprint1-2.yaml — 53 endpoint (Sprint #1 = 45, Sprint #2 = 8)

ตรวจแล้วว่า YAML ทั้งสองไฟล์ valid, ไม่มี \$ref ที่ชี้ไปยัง schema ที่ไม่มีอยู่, ไม่มี operationId ซ้ำข้ามไฟล์ และรวมกันได้ครบ **146 endpoint** ตรงกับตอนที่ 2 (อัปเดตรอบ 5 — เพิ่มจาก 142)

1. ทำไมแบ่งเป็น 2 ไฟล์ตาม MVP/Sprint แทนที่จะแบ่งตามกลุ่ม resource

พิจารณาแล้วเลือกทางนี้แทนการแบ่งไฟล์ตาม 10 กลุ่ม resource (Auth, Teams, Tournaments ฯลฯ) ด้วยเหตุผล 3 ข้อ

- ไฟล์เดียวเปิดได้จริงในเครื่องมือทั่วไปทันที** — ถ้าแบ่งตามกลุ่ม resource (10+ ไฟล์) จะต้องใช้เทคนิค cross-file \$ref เพื่อรวมเป็น spec เดียว ซึ่งต้องพึ่งเครื่องมือ bundler (เช่น swagger-cli bundle , Redocly CLI) ก่อนถึงจะเปิดใน Swagger UI ได้ปกติ — เพิ่มขั้นตอนที่ทีมต้องติดตั้งเครื่องมือเพิ่ม ในสภาพแวดล้อมที่ไม่มีเน็ตหรือเวลาจำกัด (CO-01/CO-02) ความเสี่ยงนี้ไม่คุ้ม ไฟล์ที่แบ่งตาม MVP/Sprint แต่ละไฟล์ self-contained เปิดวางในเว็บ Swagger Editor หรือ swagger-ui ได้ทันที
- ตรงกับ timeline การพัฒนาจริงตาม SRS** — MVP (R01) ส่งมอบสัปดาห์ที่ 6–9, Sprint #1/#2 (R02) สัปดาห์ที่ 10–15 ทีมพัฒนาจะเปิดใช้แค่ไฟล์ MVP ก่อนจริงๆ ไม่ต้องเห็น 53 endpoint ที่ยังไม่ต้องทำปณูอยู่
- สอดคล้องกับเอกสารชุดก่อนหน้านี้** — ตอนที่ 3/3.1 แบ่งตามขอบเขตเดียวกันมาแล้ว การคง boundary เดิมทำให้ไล่เทียบเอกสารข้ามตอนได้ง่าย (เปิด Part3 คู่กับ MVP.yaml, เปิด Part3-1 คู่กับ Sprint1-2.yaml)

ข้อแลกเปลี่ยนที่ต้องรู้ไว้: มี field/schema ที่ซ้ำกันเล็กน้อยระหว่าง 2 ไฟล์ (เช่น UserRef , TeamRef , Pagination , ErrorResponse ประกาศซ้ำ) ทำให้แต่ละไฟล์ standalone ไม่ต้องพึ่งไฟล์อื่น — ถือว่าคุ้มกว่าความซับซ้อนของการรวมไฟล์แบบ dynamic

2. จุดที่ต้องรู้ก่อนใช้งานจริง — path ที่ "ซ้อน" กันข้ามไฟล์

มี 2 path ที่ endpoint หนึ่งอยู่ใน MVP file (POST/GET) แต่อีก method อยู่ใน Sprint file (PATCH) เพราะ endpoint transition ต่างเฟสกัน

Path	Method ใน MVP	Method ใน Sprint#1/#2
/tournaments/{id}/bracket	POST (สร้าง), GET (ดู)	PATCH (จัดสายใหม่ — M03)
/matches/{id}/result	POST (ส่งผล — S01), GET (ดูผล — S05)	PATCH (แก้ผลย้อนหลัง — S08)

ถ้าจะ implement ทั้ง MVP และ Sprint#1 พร้อมกัน (หรือ deploy จริง) ต้องรวม method เหล่านี้เข้า path item เดียวกันในโค้ด backend (เช่น Express route คนละบรรทัดแต่ path เดียวกันเขียนได้ปกติ) — ไม่ใช่ปัญหาตอน implement เพราะ router ทั่วไปรองรับ multiple method ต่อ path อยู่แล้ว เป็นแค่ข้อควรรู้ตอนอ่าน spec ว่าทำไม path นี้โผล่ในทั้งสองไฟล์

ทั้งสองจุดมีคอมเมนต์ ▲ กำกับไว้ในไฟล์ YAML แล้ว

3. วิธีเปิดดูและใช้งาน

3.1 เปิดดูแบบ interactive (Swagger UI)

ออนไลน์ (ไม่ต้องติดตั้งอะไร):

- เปิด <https://editor.swagger.io>
- File → Import File → เลือก LTMS_OpenAPI_MVP.yaml
- จะเห็นทุก endpoint พร้อมปุ่ม "Try it out" ทดสอบยิงจริงได้ทันที (ถ้า backend รันอยู่และตรงกับ servers.url)

ในเครื่อง (ถ้าต้องการ host เอง):

```
npx swagger-ui-watcher LTMS_OpenAPI_MVP.yaml
```

3.2 Generate TypeScript types ให้ frontend

```
npx openapi-typescript LTMS_OpenAPI_MVP.yaml -o src/types/api-mvp.d.ts
npx openapi-typescript LTMS_OpenAPI_Sprint1-2.yaml -o src/types/api-sprint.d.ts
```

ได้ type ที่ตรงกับ schema ทุกตัวที่นิยามไว้ — frontend import ไปใช้เขียน fetch call แบบมี type-safety ได้เลย ไม่ต้องเขียน interface มือซ้ายกับที่ backend มี

3.3 Validate ว่า backend implement ตรงตาม spec

ถ้าต้องการ contract testing ที่หลัง เครื่องมืออย่าง Dredd หรือ Portman อ่าน OpenAPI ไฟล์นี้แล้วยิงทดสอบ API จริงเทียบกับ spec ได้ — เกินขอบเขตของ MVP ตอนนี้ แต่เก็บไว้เป็นตัวเลือกสำหรับ Sprint ถัดไปถ้ามีเวลา

4. สิ่งที่ยังไม่ได้ใส่ในไฟล์ YAML (จจใจ)

ไม่ได้ระบุ examples ครบทุก field — ใส่แค่ description ที่จำเป็น เพื่อไม่ให้ไฟล์ใหญ่เกินไปโดยไม่จำเป็น หน้าที่ตรงนี้ยกให้เอกสาร Part3/Part3-1 (markdown) ที่มีตัวอย่าง request/response แบบเต็มอยู่แล้ว — ให้อ่านคู่กัน

ไม่ได้ผูก error response แต่ละ endpoint กับ schema แบบเข้มงวด (ส่วนใหญ่ใช้ description บอก error code เป็นข้อความแทนการทำ oneOf เดิมรูปแบบ) เพื่อประหยัดพื้นที่ไฟล์ — รายละเอียด error แบบเต็มอยู่ใน LTMS_API_Design_Part4.md อยู่แล้ว

ไม่ได้ตั้ง rate-limit หรือ security scope แยกตาม endpoint ในระดับ spec — OpenAPI รองรับ x-rate-limit แบบ extension ได้ แต่ทีมยังไม่ได้เลือกมาตรฐานที่จะใช้ ปล่อยเป็น comment ในเอกสารเหมือนเดิม (ดู Part 0–1 หัวข้อ 1.16)

5. สรุปสถานะเอกสารทั้งหมด

ตอน	เนื้อหา	ไฟล์
0–1	Conventions + Resource Inventory	LTMS_API_Design_Part0-1.md
2	Endpoint Matrix (146 endpoint)	LTMS_API_Design_Part2.md
3	Schema — MVP (92)	LTMS_API_Design_Part3.md
3.1	Schema — Sprint #1/#2 (50)	LTMS_API_Design_Part3-1.md
4	Error Catalog	LTMS_API_Design_Part4.md
5	OpenAPI 3.1 YAML	openapi/LTMS_OpenAPI_MVP.yaml , openapi/LTMS_OpenAPI_Sprint1-2.yaml

API Design เสร็จครบทุกชั้นแล้ว ตั้งแต่ conventions จนถึงไฟล์ที่ deploy ใช้งานได้จริง สิ่งที่เหลือค้างจากทั้งกระบวนการมีแค่ 2 เรื่องที่ต้องผ่าน Change Management ก่อนใช้งานจริง (multi-faculty/external Organizer ในข้อ 4.3, และขอบเขตแก้ผลย้อนหลังในข้อ 12.2) และค่าตัวเลข sport_types.min_members / max_members ที่ต้องยืนยันกับทีม/อาจารย์ก่อน seed ข้อมูลจริง — ไม่มีข้อไหนบล็อกรการเริ่มเขียนโค้ด backend ตาม MVP ได้ทันที